

Summer Project 2026

Artefact	2
Target Audience	2
Background	2
Similar Products	3
Features	3
Timeline	3
Phases	3
Pre Development	3
Setting Up GUI	4
Adding File Handling	4
Adding Devices	4
Adding Network Capabilities	4
Post Development	4
Language	5
Java	5
C#	5
Conclusion	5
Tools	5
Git	5
Jira	5
Visual Studio Code	6
Doxygen	6
Frontend	7
Userflow	7
Main Window	8
Emulator Window	8
File UI	9
Custom Devices	9
Components	10
Microcontrollers	10
Scripts Tab	10
Libraries Required	11
JavaFX	11
Backend	12
File Handling	12
Devices	13
Running	13
Opening	14
Adding Components	15
Components	16
Microcontrollers	16

Summer Project 2026

Networking	16
Exception Manager	16
Thermal and Power Dissipation	17
Voltage and Current Stress Analysis	18
Tolerances	20
Transient and Startup Condition Tracking	22
Signal Integrity	23
Libraries Required	24
GSON	24
Jeohneliades file_explorer	25
Java.net.Socket	25
Java.Util.*	25
Pseudocode	25
Project	25
Device	26
Window	27
Exception Manager	28
Linked List	31
Behaviour Tree	32
New Project	35
Save Project	35
Open Project	35
Upload Device	36

Artefact

A desktop Hardware emulator that helps users to test their hardware virtually before buying the physical components.

Target Audience

This Application is targeted to Engineers/Programmers who work with electronics. Providing an opportunity to test their solutions before purchasing any potential products. Potentially reducing wasted money.

Background

The idea for this project came from working with Protobuf on a group project early this year, 2026. When testing the backend of our Application I needed to develop my own client to send queries. From here I had the thought of developing an application for testing these electronic device clients that would provide an easy user experience.

Summer Project 2026

Similar Products

<https://www.tinkercad.com/circuits>

Tinkercad provides a very similar experience to what I aim to develop; however, it also includes a blueprint programming system to develop programs within the app. My application will fit in to provide greater control to the user and networking capabilities. This means that more professional or dedicated hobbyists will be more interested in the application due to the greater freedom provided.

https://www.edrawsoft.com/ad/electrical-drawing-software.html?gad_source=1&gad_campaignid=11142611514&gclid=Cj0KCQjw_7PRBhDcARIsAMjV7jmD23TAbozRfi4Ji2elsPRuvn5MuFD3wQK10Kmxefs-zNdDL0WEff0aAqilEALw_wcB

EDrawSoft provides a free electronic schematic designer application. Within, users can design and test their schematics. However they do not include Microcontrollers which gives my application an edge over EDrawSoft. This means that my application can cover a wider range of use scenarios therefore increasing the number of potential users.

Features

- Create Custom Devices
 - Users upload config files outlining the hardware component. The emulator will read the file and add it to a list.
- Graphical User Interface (GUI)
 - A GUI is implemented to produce an easier experience for the user.
- Websocket Support
 - Emulated hardware can communicate with websocket servers using Protobuf. To increase the range the app can cover

Timeline

The timeline will be split up into separate phases each covering different parts of the development process. Phases were chosen to help temper expectations throughout the development period and keep on track of the goal. Furthermore these goals can be assured to be achievable not setting unrealistic expectations meaning a less mentally taxing development process.

Phases

Pre Development

26th May to 17th June.

Summer Project 2026

During the Pre Development period planning will occur, covering every detail of the artefact. This means that I do not get lost in development and know what to do before beginning development.

This Phase will also include setting up all the software and solutions to be ready for development

Setting Up GUI

17th June to 15th July.

The GUI window will be created first to make easy testing of additions in the future as I can physically see the results of my programs.

Adding File Handling

15th July to 12th August.

File Handling is a key part of any App therefore will be implemented next. Creating save files for the user will be contained here and this phase follows well into the next.

Adding Devices

12th August to 9th September.

The ability to upload config files to the application will be covered, here allowing the user to add their custom controllers.

A components sidebar will also be developed for creating devices within the window

Adding Network Capabilities

9th September to 7th October.

This phase will provide the ability for controllers to communicate over the network.

Post Development

7th October to 18th October.

Finally this phase is responsible for polishing and testing the artefact.

Summer Project 2026

Language

Java

Java is typically used for mobile/desktop/web applications making it suitable for the artefact. The language is object oriented, features many libraries due to it being an older language, supported by Doxygen making documenting very easy and finally platform independent making a portable solution.

C#

I already have experience developing UI and using C# for a similar artefact. However I would like to branch out and work with different languages. Previously I had used Avalonia to develop a UI frontend having a fairly easy experience.

Conclusion

I intend to use Java to further grow my skills and experiences.

Tools

Git

Git will be used for backing up the artefact and segmenting development phases. This is because of Git's branching capabilities meaning that users can modify the artefact without affecting the other developers code. Although I am developing this artefact independently, branches can be used to further outline the phases keeping me on track.

I have chosen to use Git over Github because I would like to learn the software and believe the experience gathered will aid me in my career. This will mean experiencing a learning curve, which I will need to keep in mind when developing so as not to effect deadline or personal motivation.

Jira

Jira will be used for project management due to it providing many useful tools to map out tasks and progress, such as a Kanban Board. In addition, Jira is frequently used within the industry meaning that I will have an easier time onboarding with companies in the future.

Whilst I did attempt to use Jira for my dissertation, I stopped using it midway through the development process meaning that I lost track of overall progress and deadlines. In this project I need to make sure to continually use the software. This will be done by making many smaller tasks so as to frequently return to Jira, keeping it fresh in my mind.

Summer Project 2026

[Link to Jira Kanban board](#)

Visual Studio Code

VS Code shall be the Integrated Development Environment (IDE) of choice due to the easy terminal support making compilation of Java classes easy. Furthermore the IDE features many extensions to ease development and debugging.

VS Code is preferred over IDEs such as Rider because there are minimal differences between the two and I generally prefer working within VS Code making the development process more enjoyable as I will not have to fight the IDE to figure out certain actions.

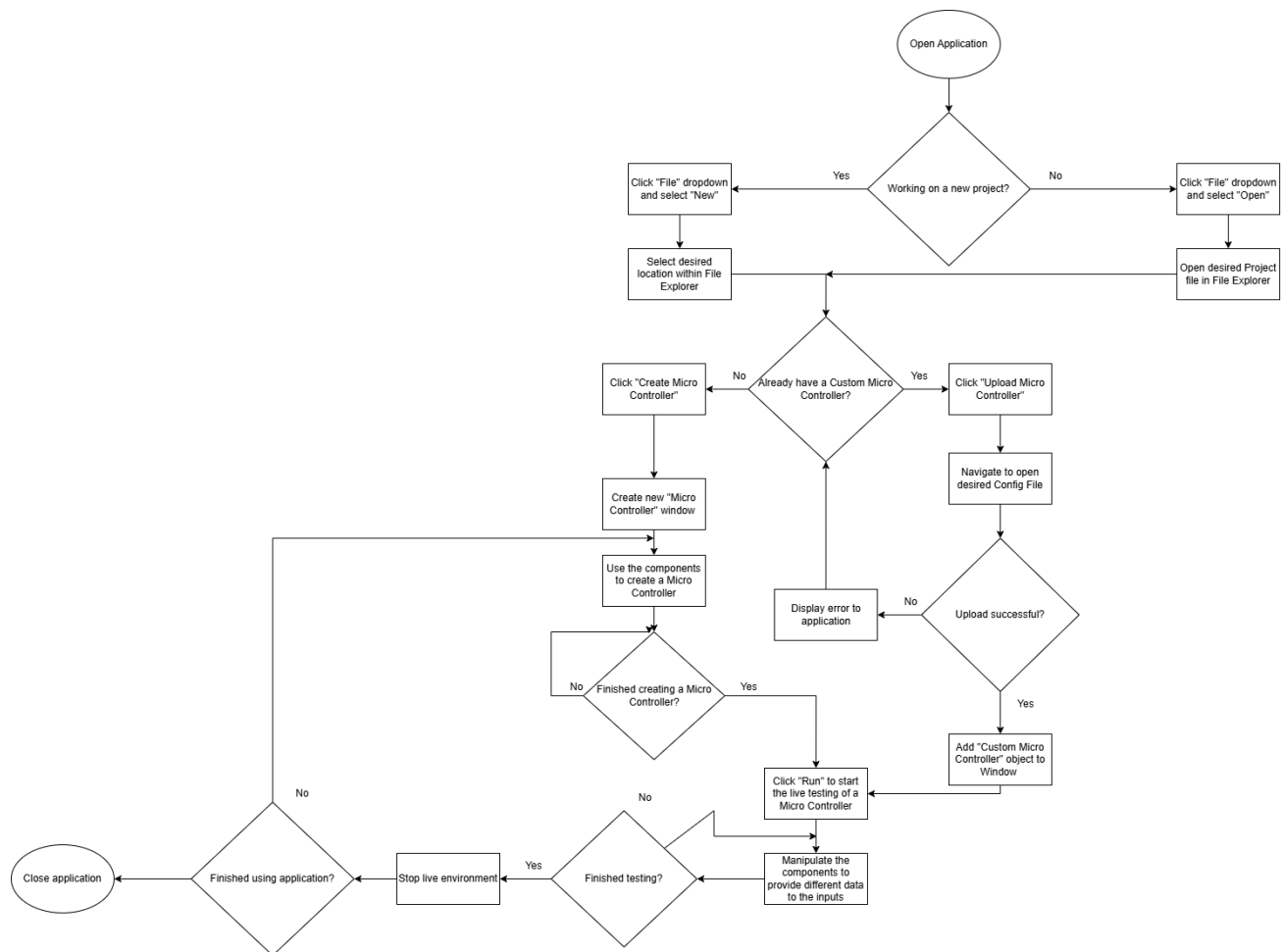
Doxygen

Doxygen is the chosen documentation tool because it supports Java files and features a tool for automatic generation of technical docs making it a great solution. Furthermore it outputs the Docs in 2 formats: HTML and Latex PDF. This means that it is accessible to users regardless of internet connectivity.

Summer Project 2026

Frontend

Userflow

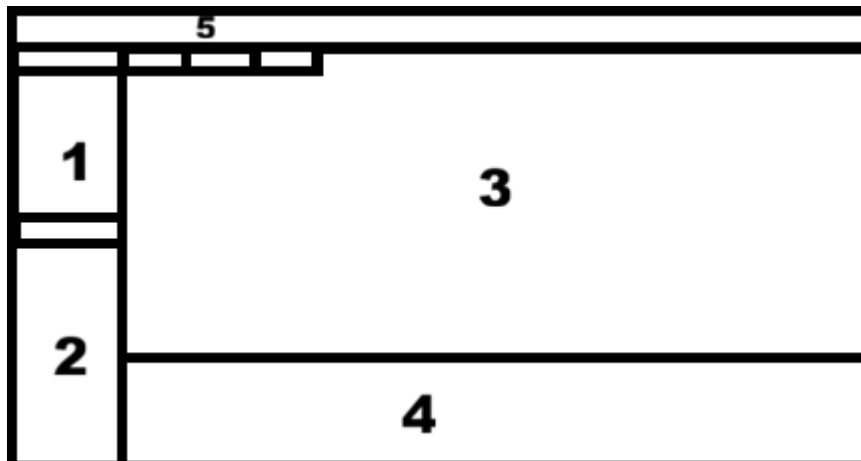


Flowchart depicting general Userflow of the application

The flowchart above aims to depict the flow of how a user might use the app. This gives an insight to key features required to provide a comfortable experience, as without such could provide friction between the user and achieving their goals. Furthermore this informs me of what may need to be developed first in terms of importance to the overall application. An example of this could be the uploading of Configuration files. This is because the Custom Devices could be made in the app, meaning it is more of a quality of life feature as opposed to the individual components used to create Devices within the app.

Summer Project 2026

Main Window



Mock up of the planned structure of the GUI for the application

Above is a rough image of the planned structure of the application the controls numbered entail:

1. Devices
2. Components
3. Emulator Window
4. Console
5. Application Features

<https://www.figma.com/make/9exi6JchvVqzocOU5rP0ZO/Microcontroller-Emulator-App?p=f&t=kfzCvSdOyTgCe2Cy-0>

Link to Figma application

Figma was used to create an AI concept of the GUI with how the user can interact with the app. This helps me to quickly tweak the actions the user can make finding the features a user may want without market research.

Emulator Window

The Emulator Window will act as a Printed Circuit Board (PCB). Here the user may drag and drop Components from their tab to the workspace. To connect components together is a pencil icon where tracks can be drawn between Components. If a mistake were to be made an "Eraser," feature will be present too.

Testing for Cross-talk by the Exception Manager will be required to assure the drawn tracks are not too close.

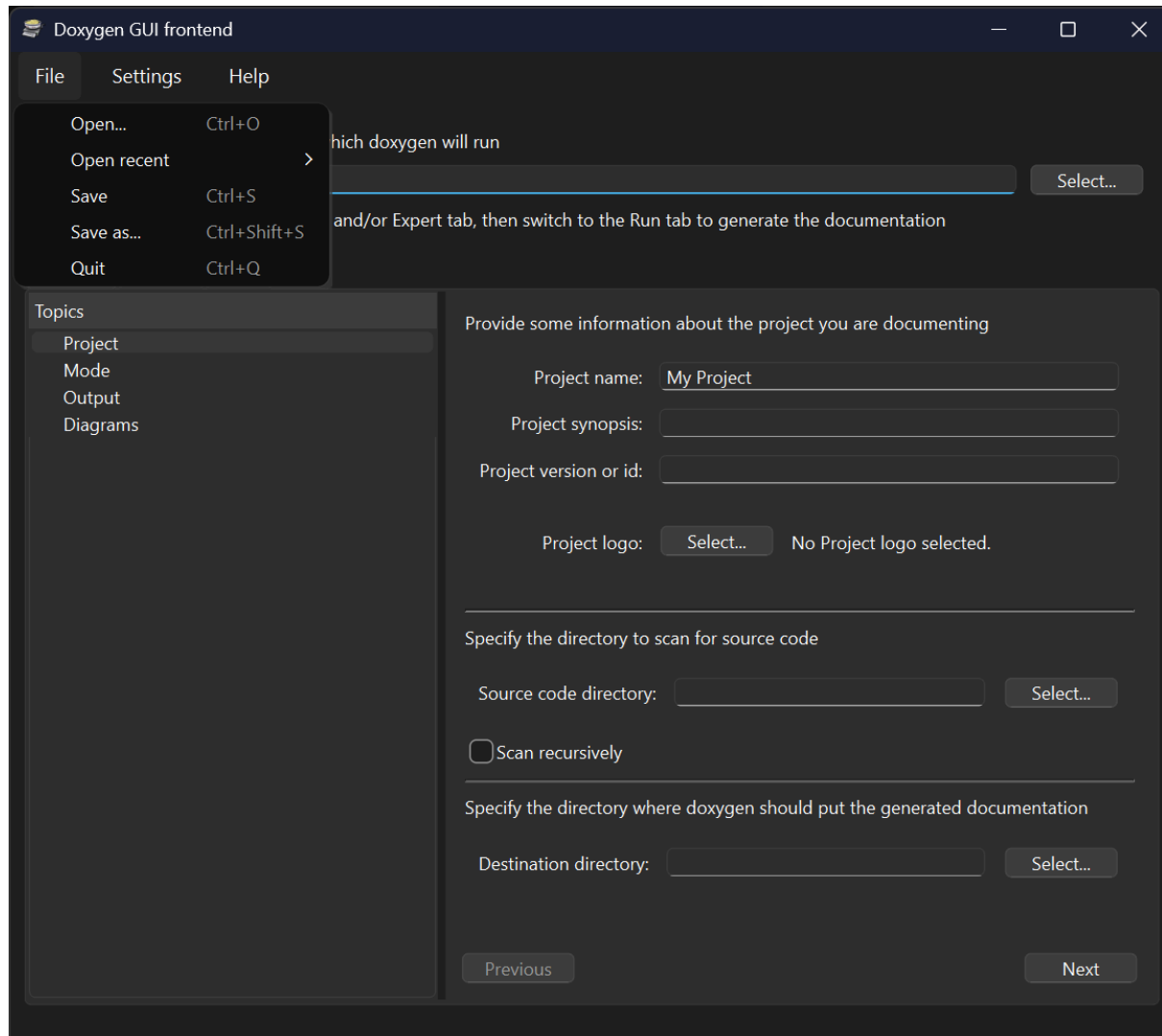
A Key will be required to inform the user of the scale at which they are working, to avoid misunderstandings in physical dimensions. For example the size of components or the board.

Summer Project 2026

File UI

The GUI for the file system will consist of a dropdown providing the options to:

- Create a new project
- Save open project
- Open a project
- Upload a custom device



Doxygen app GUI for file handling

Above is an example of how I would like the UI for file handling to look like. This is an intuitive design already used by many other applications within software development.

Custom Devices

Within the file dropdown is an “Upload Device,” button. This can be used to upload a premade custom device to be used within the app. Once successfully uploading a Custom Device it will appear in the devices tab which can be opened in a new Emulator Window. If unsuccessful an error will be logged to the terminal.

Summer Project 2026

Double clicking a Device within the “Devices,” tab will open a new window within the emulator containing its structure and components. From here the User may modify and test the Device.

Components

Housed within the sidebar, alongside Devices, will be a “Components,” tab. Here all the required Components to build a Device within App will be stored.

By dragging from the tab and dropping within the emulator window a Component can be added. These components may be connected to a Microcontrollers’ pin by dragging from the pin of one and releasing over the pin of the other. This will allow for Users’ to test more complex devices, improving the range of scenarios.

Right-clicking a component will display its details featuring Current, Resistance and Voltage. These variables will be Text Blocks adjustable by the User to test all possible scenarios. This means that Devices can be thoroughly tested by the User.

Microcontrollers

Contained within this tab will be Microcontrollers. These will behave similarly to the other components within the GUI however also provide the ability to receive a Java file from the User. This means that a Microcontroller's pseudocode can be tested on emulated hardware simultaneously to the hardware of their proposed device.

Scripts Tab

To supply a microcontroller with a user generated script, the app must accommodate the ability to upload “.Java,” files. Similarly to Devices and Components, when pressing the “Upload Script,” button File Explorer will open. From here the User may navigate and select the desired file. If successful a new “Script Card,” will be instantiated containing the path to the file.

A “Script Card,” may be dragged from the tab and dropped on a placed Microcontroller, providing a visual confirmation and passing the contained file path.

This visual design provides an intuitive user experience through implementing a card structure, meaning that users spend less time learning how to use the application. Therefore featuring a faster rollout.

Summer Project 2026

Libraries Required

JavaFX

JavaFX is a library for creating applications within Java. It has been used for many applications by the community, capable of a wide range of designs.

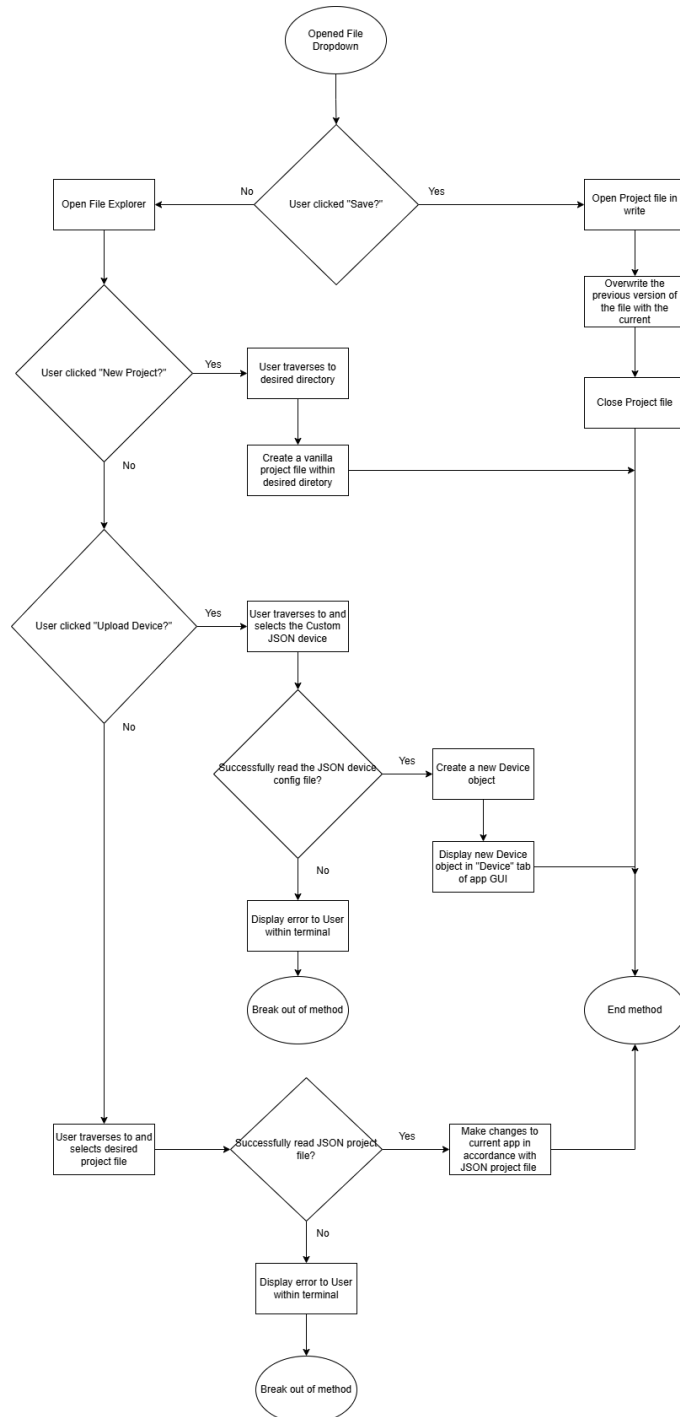
This library was preferred over others such as Swing due to it being actively maintained. This means that JavaFX features up-to-date documentation, modern feature support for example CSS files and not feature frozen. These result in a future proofed solution, receiving support from active developers.

https://fxdocs.github.io/docs/html5/#_introduction

Summer Project 2026

Backend

File Handling



Flowchart depicting the process to handle files

The flowchart above depicts how the backend will handle files. Flowcharts allow me to avoid getting lost in the details of everything I need to do. This abstraction helps to set easily understandable goals quickly and model the desired process. Furthermore peers who may view this document can discern my thought process and the plans to reach my goal.

Summer Project 2026

Devices

Running

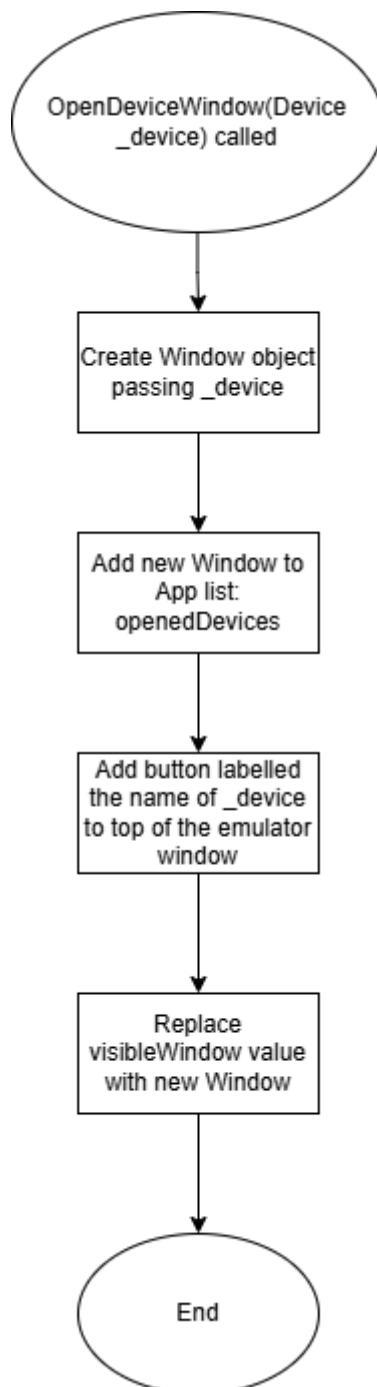
Running will consist of stepping through a linked list of components, breaking and logging to the console if an error were to occur. The use of a linked list allows for the implementation of parallel circuits through a boolean value controlling the transition between states. An example of this would be providing a “Node,” parent class for series circuits and a “Switch Node,” child class for switches allowing for the process to follow the selected path.

Whilst running the User can adjust the values of components to test how the Device performs under different circumstances.

Finally, before beginning running the application will check if a power source is present within the circuit of the device. A power source will be required because I intend to begin the running process from this component.

Summer Project 2026

Opening



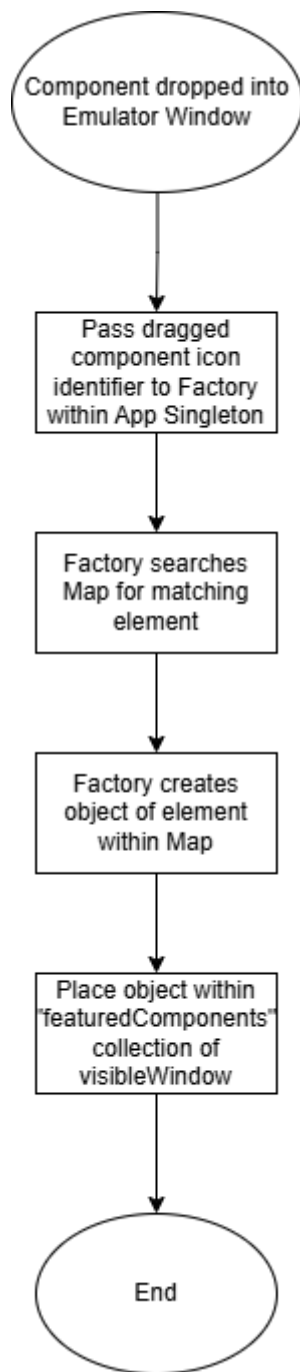
Flowchart following the process to open a Device Window

A function will be responsible for opening a new Device window. A new Window object will be created containing the parsed Device object. This new object will be added to the GUI and a list of opened devices where the user may modify or run.

Separate Window objects allow for easy deletion of undesired windows and can isolate featured components. This avoids potential errors involving components bleeding between different Device windows.

Summer Project 2026

Adding Components



Flowchart depicting process for adding Component to Emulator Window

When dropping a component from the sidebar into the emulator window, a factory is used to create the relevant Component object within the currently visible window. Using a Factory means that all instantiation can be limited to a single class making debugging easier as only a single file needs to be reviewed.

An icon and identifier is only used when dragging from the sidebar to avoid creating an object only for the user to cancel the action. This should improve efficiency as objects are only instantiated when the User confirms their choice.

Summer Project 2026

Components

Components will be modelled using Behaviour Trees as these work in hand with the linked list used for running a device. Furthermore these provide the flexibility required for the range of components featured. Behaviour Trees feature an “Exit,” state which can be used to call the next element of the linked list.

Behaviour Trees were chosen to model components because of their modularity. In this scenario having a modular system is imperative for creating an efficient application, due to the large array of components. This means that components can be created quickly as opposed to hard coding parent and child classes.

Microcontrollers

Microcontrollers will be modelled using Finite State Machines (FSMs). This is because Microcontrollers typically do not have a set order of operations and can progress dynamically.

Users' must code the FSM and individual states independently as I cannot write a system capable of accommodating all uses. However I can accommodate their FSMs best as possible by programming the application to treat the Microcontroller as an FSM and avoid crashes if relevant errors are encountered.

Networking

Java provides many different libraries to accommodate networking ranging from TCP to Websockets.

In this artefact Websockets will be used to facilitate Networking. This is because Websockets provide a low latency, persistent two-way communication solution. This is appropriate for the application as the device needs to be simulated alongside handling online communications.

The application will act as the Host for the Websocket server facilitating clients' communications, and the simulated device as a client. This allows for the User to include networking capabilities in their Devices. Therefore improving the reach of the application.

Exception Manager

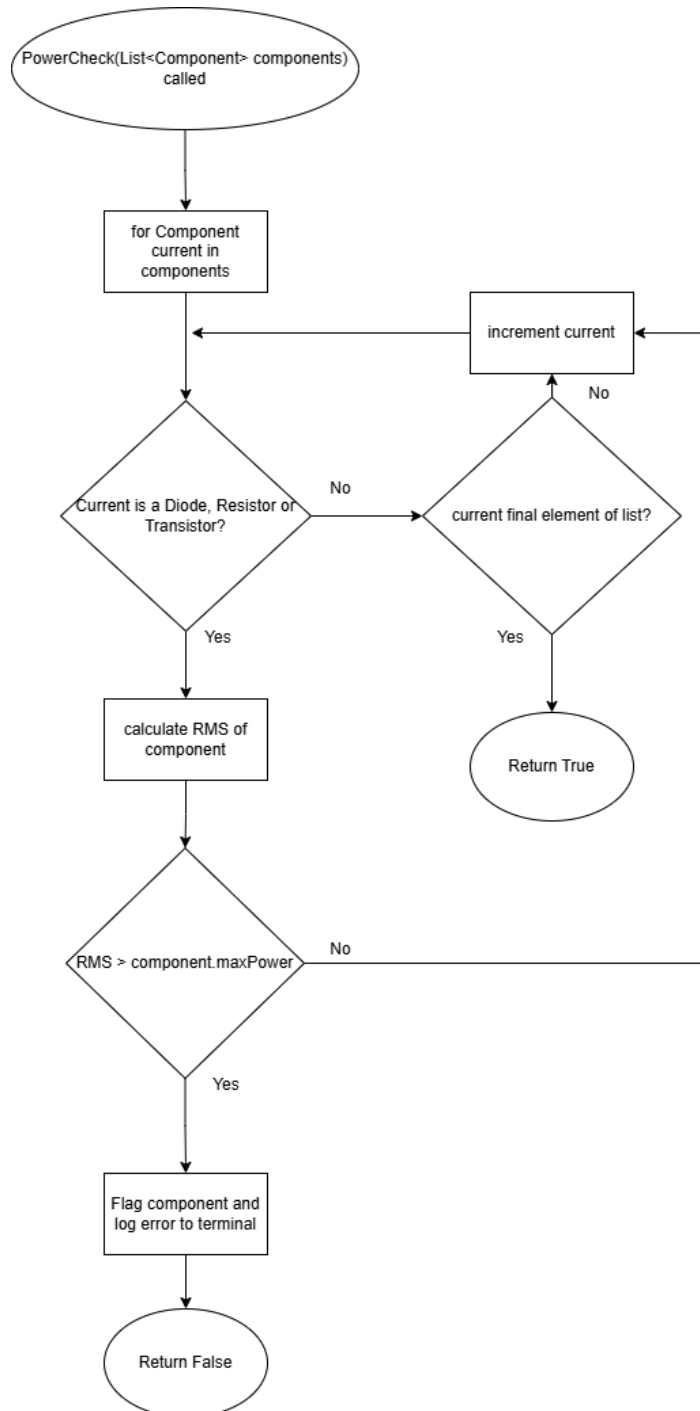
The Exception Manager will be a Singleton responsible for discerning the health of the circuit/components during simulation. A singleton will help to improve modularity meaning if a problem were to arise it would originate from a single implementation.

Exception Managers' functions will be called from the component Behaviour Tree Decorator Nodes to assure the solution is safe before continuing to the following component. These functions can be called from anywhere and easily placed into the condition of a Conditional Node. Through using inheritance, carrying out a health check before executing the component can be applied to all future components.

Summer Project 2026

Thermal and Power Dissipation

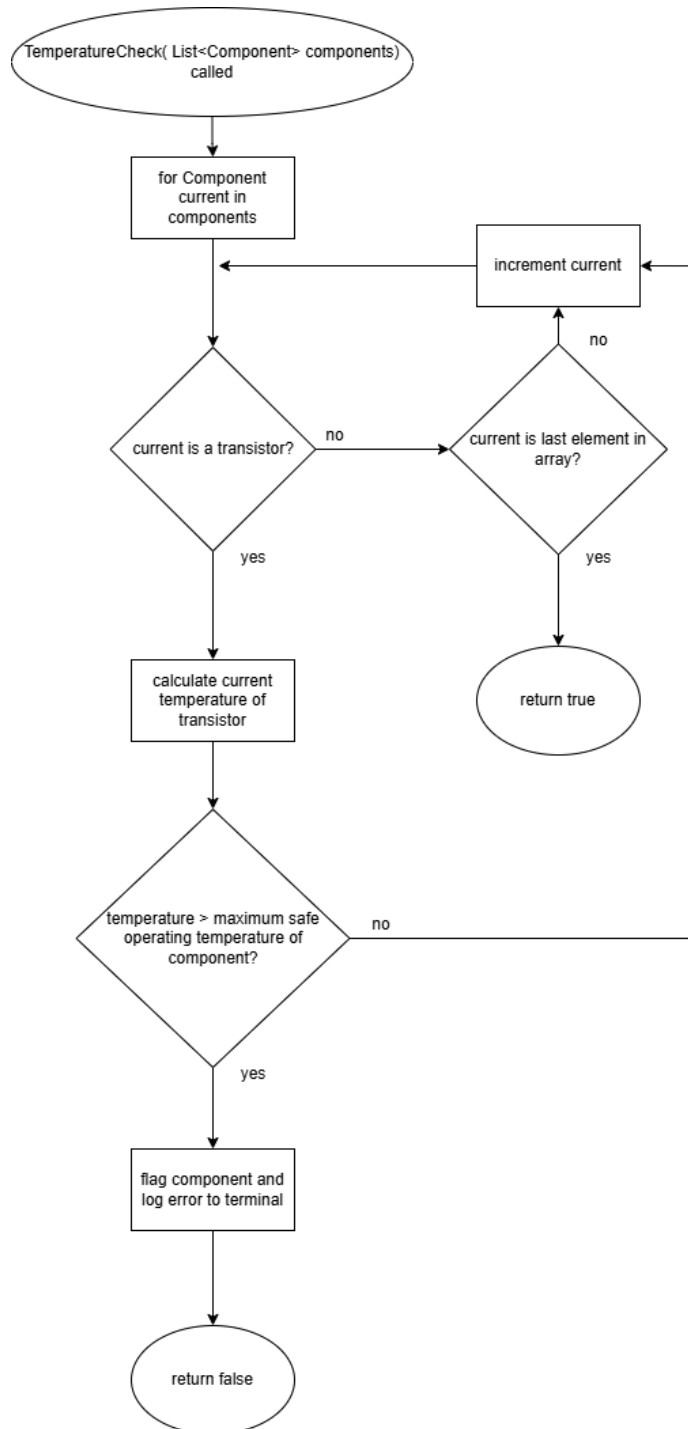
The Root-Mean-Square (RMS) of diodes, resistors and transistors will be calculated to check for power dissipation. If the RMS is greater than the maximum power of the component then it shall be flagged and the reason displayed.



Flowchart depicting the function responsible for checking power dissipation

Summer Project 2026

The temperature of transistors will be calculated to avoid reaching temperatures capable of melting the component. Similar to the previous test the component shall be flagged citing the reason.

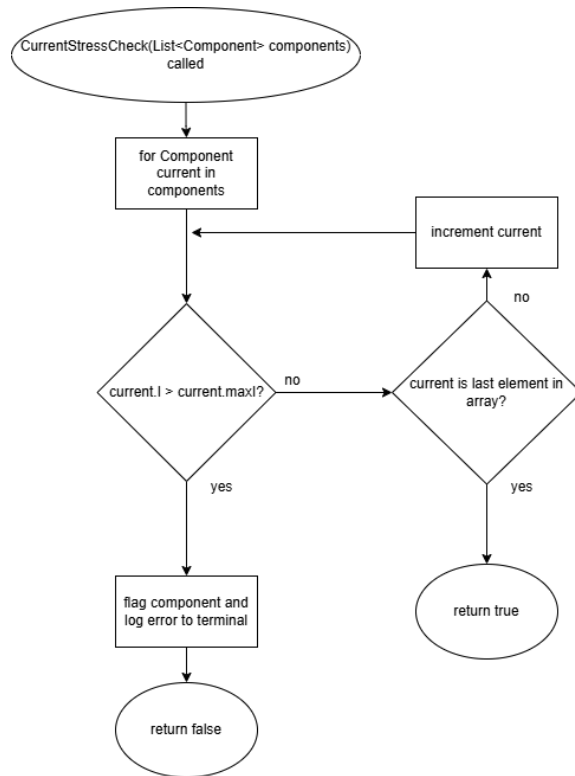


Flowchart depicting the function for checking temperature dissipation

Voltage and Current Stress Analysis

The peak current will be calculated during the starting and running stages. This is to test for stress on the circuit from large currents. Tracks and components affected the most by the stress shall be flagged, bringing the users attention where it is most needed.

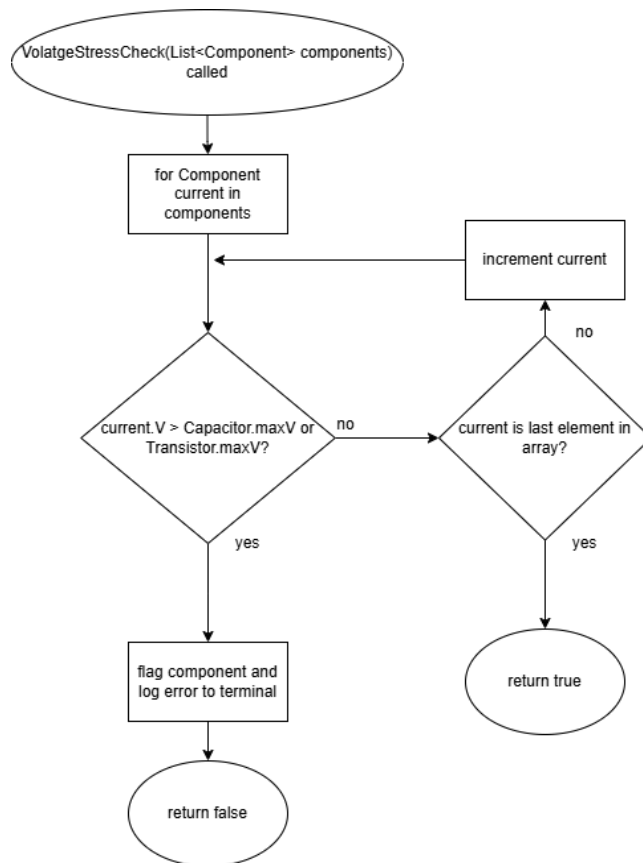
Summer Project 2026



Flowchart depicting function responsible for checking current stress

Voltage of capacitors and transistors will be calculated to avoid stress on components further along the tracks. This will be done by calculating the max voltage the components are under, when being switched on. A flag will appear over the affected components.

Summer Project 2026

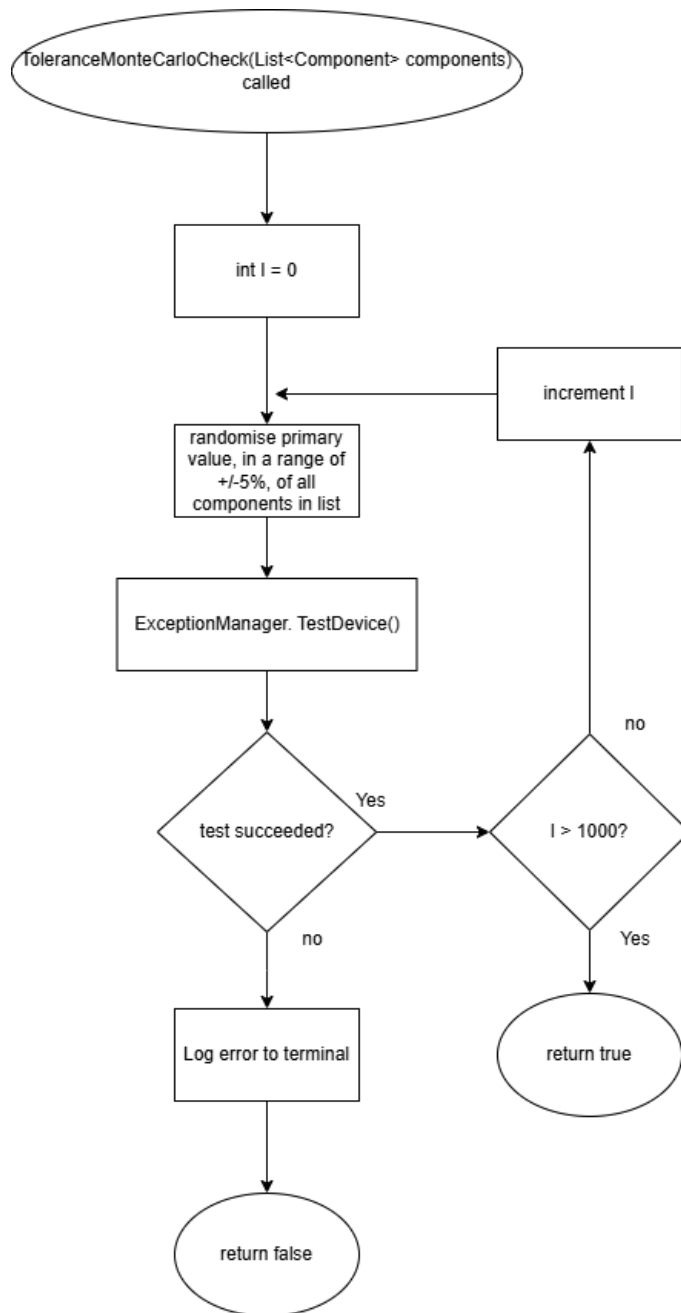


Flowchart depicting function responsible for checking voltage stress

Tolerances

Tolerances are the variances in manufactured components. To assure these tolerances do not affect the device a Monte Carlo simulation will be run at the beginning of testing. The primary value of components will be randomized between a $\pm 5\%$. Any components that fail during this process will be flagged.

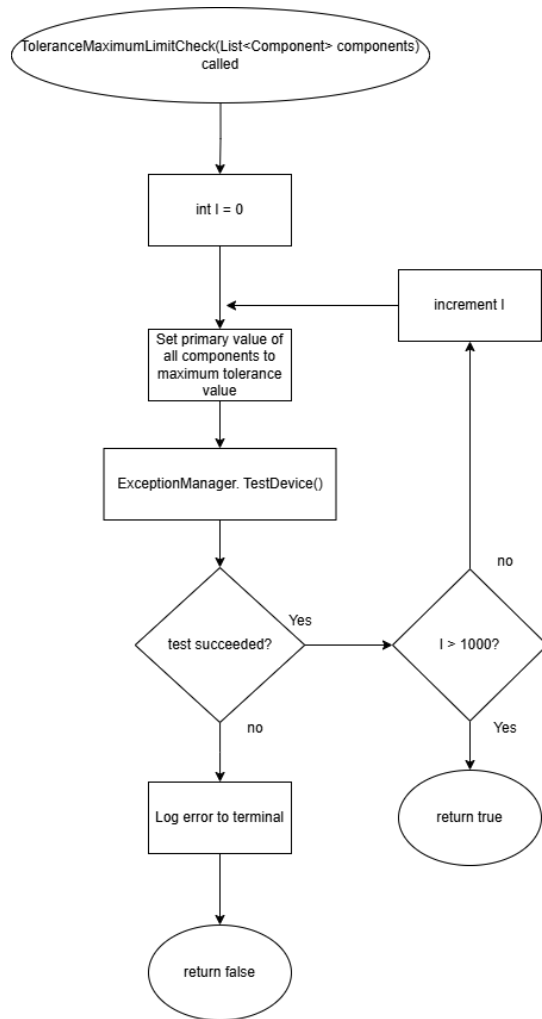
Summer Project 2026



Flowchart depicting function for Monte Carlo testing tolerances of components

Another tolerance test will be simulating all components to reach their maximum tolerance. This will check that the device can operate safely if at the limit of its tolerance, flagging any components that fail.

Summer Project 2026



Flowchart depicting function responsible for checking the result of all components at maximum tolerance

Transient and Startup Condition Tracking

When a switch opens turning off a motor, the presence of a flyback diode needs to be checked for. This is to avoid voltage spikes from the collapsing magnetic field. When such conditions are met the motor will be flagged displaying the issue.

Summer Project 2026



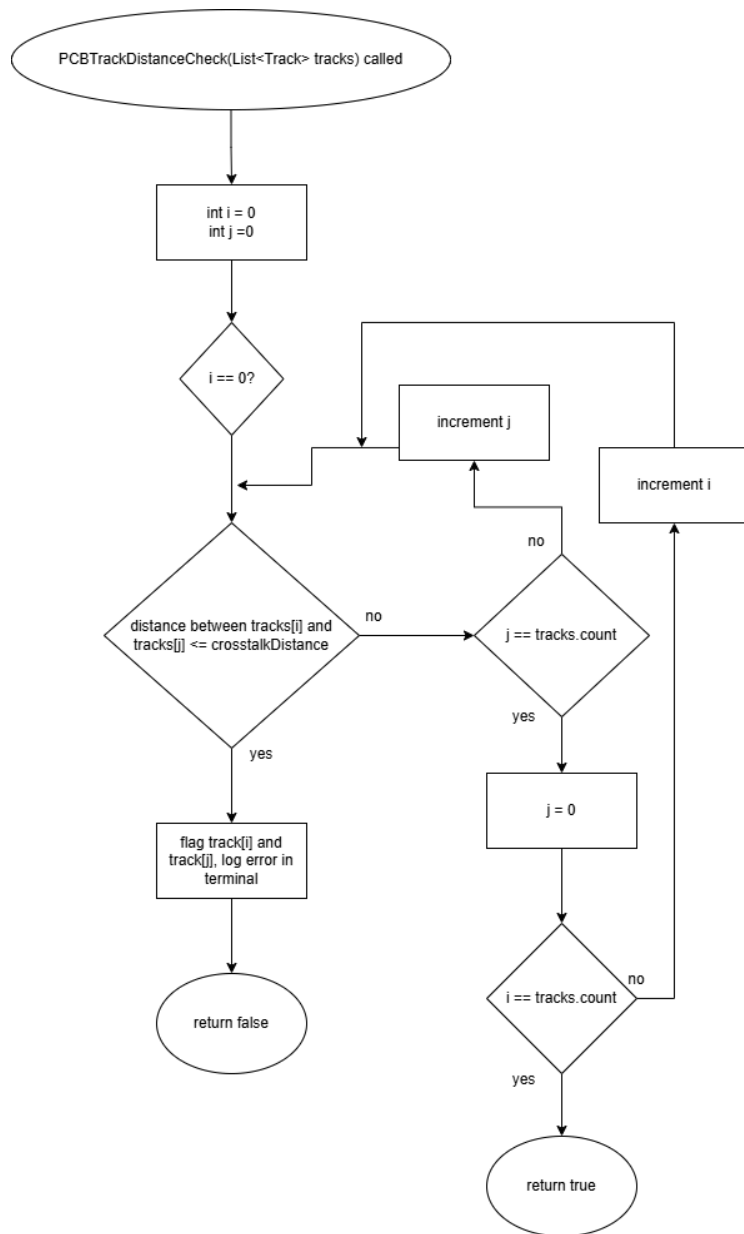
Flowchart depicting function responsible for checking if a flyback diode is present when a switch turns a motor off

Upon startup uncharged capacitors can act like short circuits drawing a large current affecting components upstream. The Exception Manager will need to test whether components will be damaged.

Signal Integrity

To avoid crosstalk between tracks on the PCB, checking that 2 tracks are not too close will be required. Tracks that meet the conditions to trigger the error will be flagged and the reason displayed.

Summer Project 2026



Flowchart depicting function responsible for checking tracks are not too close to each other

Destructive signal reflections can occur when track widths do not match the source and load impedance. This will not need to be tested for though as a system will be in place forcing only certain track widths to be used.

Libraries Required

GSON

Whilst Java does have the ability to read files using the class "[Java.IO.File](#)," this does not include JSON files which is my desired format. Therefore I will use GSON, a Java library used to convert Java Objects to JSON and vice versa. This means that I can write and read to/from Users' JSON project and Device Configuration files.

Summer Project 2026

GSON was preferred over other libraries, such as Jackson, because I am not using a JAX-RS or Spring Framework as I am using OpenJDK. Furthermore GSON has simpler To/FromJSON methods as they provide both Simple and Custom serialization and deserialization.

Incorporating GSON into my application is quite straightforward, as I only need to add the following to my Gradle dependencies: `implementation`

`'com.google.code.gson:gson:2.11.0'`. This means that I can quickly implement the module and continue working on the application.

Jeohneliades file_explorer

This library provides a file explorer application that can be opened from the Java application. Meaning that I can provide a file explorer for the User to navigate and find their desired projects or devices.

A drawback of this library is that it will require JavaFX however I already intend on using JavaFX for my application UI making it a suitable fit.

Java.net.Socket

This library provides the fundamentals to create Socket operations, which will be required to enable the User to implement networking capabilities to their devices. A large array of functions and classes are provided allowing for a simple and flexible setup of a Websocket server.

Java.net.Socket is preferred over others due to it already being present within Java. This means I do not need to download any external libraries.

Java.Util.*

The Java.Util library will be required to implement key data structures for development of the application, for example linked lists. This library is already featured within Java meaning it does not need to be downloaded from an external source, making implementing simple.

Pseudocode

Project

```
Public class Project
{
    Device[] devices;
```

Summer Project 2026

```
Window[] openWindows;

Public Project(Device[] _devices, Window[] _openWindows)
{
    devices = _devices;
    openWindows = _openWindows;
}

Public Project(){}
}
```

Device

```
Public class Device
{
    list<Component> featuredComponents;
    Enum deviceType;
    String iconPath;
    LinkedList circuit;

    Public void Run()
    {
        Component currentNode = circuit.getFirst();
        While True
        {
            currentNode.component.Run();
        }
    }

    Public void Save()
    {
        App.SaveDevice(this);
    }

    Public void AddComponent(Component newComponent)
    {
        featuredComponents.Add(newComponent);
    }

    Public void RemoveComponent(Component removalComponent)
    {
        featuredComponents.Remove(removalComponent);
    }

    Public void AddToCircuit(Component newComponent, int index)
```

Summer Project 2026

```
{
    circuit.Add(index, newComponent);
}
Public void RemoveFromCircuit(Component removalComponent, int index)
{
    Circuit.Remove(index, removalComponent);
}

Public Device();

Public Device(string _iconPath, enum _deviceType)
{
    iconPath = _iconPath;
    deviceType = _deviceType;
}
}
```

Window

```
Public class Window
{
    Device device;
    Float scale;
    Array<float> viewPosition;

    Public void SetViewPosition(Array<float> newPosition)
    {
        viewPosition = newPosition;
    }

    Public void SetScale(float newScale)
    {
        scale = newScale;
    }

    Public Window();

    Public Window(Device _device, float _scale, float _viewPosition)
    {
        device = _device;
        scale = _scale;
        viewPosition = _viewPosition;
    }
}
```

Summer Project 2026

Exception Manager

```
Public class ExceptionManager
{
    Private static ExceptionManager* current;

    Public static ExceptionManager& Current()
    {
        if(!current) { current = new ExceptionManager(); }
        Return *current;
    }

    Public bool PowerDissipationCheck(List<Component> components)
    {
        foreach(Component current in components)
        {
            if(current.type == Resistor)
            {
                Float Irms = current.Ipeak / sqrt(2);
                Float p = Pow(Irms,2) * current.R;
                if(p >= current.maxP)
                {
                    App.Flag(current);
                    App.Terminal("Power dissipation failure");
                    Return false;
                }
            }
            Else if (current.type == Diode || Transistor)
            {
                Float Pavg =  $\frac{1}{T} \int_0^T (v(t) * i(t)) dt$ ;
                if(Pavg > current.pMax)
                {
                    App.Flag(current);
                    App.Terminal("Power dissipation failure");
                    Return false;
                }
            }
        }
        Return true;
    }
}
```

Summer Project 2026

```
Public bool TemperatureDissipationCheck(List<Component> components)
{
    foreach(Component current in components)
    {
        if(current.type == Transistor)
        {
            Float tempJunc = current.tempAmb + (current.P *
current.thermalResistance;
            if(tempJunc > current.meltingPoint)
            {
                App.Flag(current);
                App.Terminal("Thermal dissipation failure");
                Return false;
            }
        }
    }
    Return true;
}

Public bool CurrentStressCheck(List<Component> components)
{
    foreach(Component current in components)
    {
        if(current.I > current.maxI)
        {
            App.Flag(current);
            App.Terminal("Current induced stress failure");
            Return false;
        }
    }
    Return true;
}

Public bool VoltageStressCheck(List<Component> components)
{
    foreach(Component current in components)
    {
        if(current.V > capacitor.maxV || transistor.maxV)
        {
            App.Flag(current);
            App.Terminal("Voltage induced stress failure");
            Return false;
        }
    }
    Return true;
}
```

Summer Project 2026

```
Public bool ToleranceMonteCarlo(List<Component> components)
{
    for( int i = 0; i < 1000; i++)
    {
        List<Component> randComps = for Component c in components
{return c.SetPrimaryValue(random(c.GetPrimaryValue(),±5%));}
        if(!ExceptionManager.TestComponents(randComps))
        {
            App.Terminal("Failure due to tolerances");
            Return false;
        }
    }
    Return true;
}

Public bool ToleranceMaximumLimitCheck(List<Component> components)
{
    List<Component> maxComps = for Component c in components{ return
c.SetPrimaryValue(c.GetPrimaryValue().Max());}
    if(!ExcpetionManager.TestComponents(maxComps))
    {
        App.Terminal("Failure due to upper limit of tolerances");
        Return false;
    }
    Return true;
}

Public bool ParallelFlybackDiodeCheck(Node motor)
{
    if(motor.parallelNode)
    {
        if(motor.parallelNode.Type == FlybackDiode)
        {
            App.Flag(motor.component);
            App.Terminal("NoFlyback Diode in parallel with motor");
            Return false;
        }
    }
    Return true;
}
```

Summer Project 2026

```
Public bool PCBTrackDistanceCheck(List<Track> tracks)
{
    for( int i = 0; i < tracks.Count; i++)
    {
        for(int j = 0; j < tracks.Count; j++)
        {
            if(tracks[i].position - tracks[j].position >=
crosstalkDistance)
            {
                App.Flag(motor.component);
                App.Terminal("Tracks too close causing crosstalk");
                Return false;
            }
        }
    }
    Return true;
}
}
```

Linked List

```
Public Node
{
    Component component;
    Node next;

    Public Node();

    Public Node(Component _component, Node _next)
    {
        component = _component;
        next = _next;
    }
}

Public SwitchNode : Node
{
    Override Array<Node> next;

    Public SwitchNode();
}
```

Summer Project 2026

```
Public SwitchNode(Component _component, Array<Node> _next)
{
    Component = _component;
    Next = _next;
}

Public void DecideNext(boolean choice)
{
    if(choice){ Next[0] }
    Else{ Next[1] }
}
}
```

Behaviour Tree

```
NodeStatus { SUCCESS, RUNNING, FAILURE };

Public BTNode
{
    Public Virtual NodeStatus Execute();
    Public virtual void Reset();
    Blackboard bb;

    Public BTNode(_bb)
    {
        bb = _bb;
    }
}

Public BTCompositeNode: BTNode
{
    List<BTNode> children;
    Int currentNode = 0;

    Public override virtual void Reset()
    {
        currentNode = 0;
        foreach( var child in children)
        {
            child.Reset();
        }
    }
}
```


Summer Project 2026

```
    Public void AddChild(Node child)
    {
        children.Append(child);
    }

    Public BTCompositeNode(Blackboard _bb)
    {
        BTNode(_bb);
    }
}

Public DecoratorNode: BTNode
{
    Node decoratedNode;

    Public virtual override void Reset()
    {
        decoratedNode.Reset();
    }

    Public DecoratorNode(Blackboard _bb, Node decorating)
    {
        BTNode(_bb);
        decoratedNode = decorating;
    }
}

Public ConditionalNode: DecoratorNode
{
    Public virtual NodeStatus condition();
    Public virtual override NodeStatus Execute()
    {
        if(condition)
        {
            Return decoratedNode.Execute();
        } else {
            Return NodeStatus.FAILURE
        }
    }

    Public ConditionalNode(Blackboard _bb, Node decorating)
    {
        DecoratorNode(_bb, decorating);
    }
}
```

Summer Project 2026

```
Public InverterNode: DecoratorNode
{

    Public virtual override NodeStatus Execute()
    {
        NodeStatus result = decoratedNode.Execute();
        if(result == NodeStatus.SUCCESS)
        {
            Return NodeStatus.FAILURE;
        } else if(result == NodeStatus.FAILURE) {
            Return NodeStatus.SUCCESS;
        } else { return result; }
    }

    Public InverterNode(Blackboard _bb, Node decorating)
    {
        DecoratorNode(_bb, decorating);
    }
}

Public Sequence: BTCompositeNode
{
    Public virtual override NodeStatus Execute()
    {
        NodeStatus result = children[currentNode].Execute();
        if(result == NodeStatus.FAILURE)
        {
            Reset();
            Return NodeStatus.FAILURE;
        } else { return result }
    }

    Public Sequence(Blackboard _bb)
    {
        BTCompositeNode(_bb);
    }
}

Public Selector: BTCompositeNode
{
    Public virtual override NodeStatus Execute()
    {
        NodeStatus result = children[currentNode].Execute();
        if(result == NodeStatus.SUCCESS)
        {
            Reset();
        }
    }
}
```

Summer Project 2026

```
        Return NodeStatus.SUCCESS;
    } else { return result }

    Public Selector(Blackboard _bb)
    {
        BTCompositeNode(_bb);
    }
}
```

New Project

```
Public void CreateNew(string dirPath)
{
    Project newProject = new Project();
    GSON gson = new GSON();
    gson.toJson(newProject, new FileWriter(dirPath));
    App.current.OpenProjectFile(newProject);
}
```

Save Project

```
Public void SaveProject(string projectPath)
{
    GSON gson = new GSON();
    gson.toJson(App.current.currentProject, new FileWriter(projectPath));
}
```

Open Project

```
Public void OpenProject(string projectPath)
{
    GSON gson = new GSON();
    Project desiredProject = gson.fromJson(
        new FileReader(projectPath),
        Project.class);
    App.current.OpenProjectFromFile(desiredProject);
}
```

Summer Project 2026

Upload Device

```
Public void UploadDevice(string devicePath)
{
    GSON gson = new GSON();
    Device desiredDevice = gson.fromJson(
        New FileReader(devicePath),
        Device.class);
    App.current.AddDevice(desiredDevice);
}
```